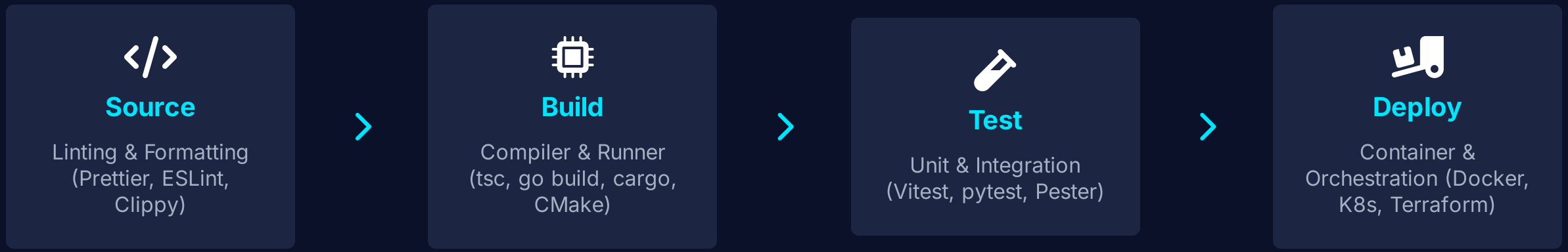
Technologie- und Tooling-Übersicht

BELASTBARE STACKS FÜR MODERNE SOFTWAREENTWICKLUNG

Der gemeinsame Werkzeugkasten



JavaScript/TypeScript und Go

JS / TS Stacks

Runtime:
Node.js, tsx, Vite

Build/Typen:
TypeScript (tsc), package.json

Qualität:
ESLint, Prettier

Testing:
Vitest, Playwright

Go Ecosystem

Core:
Go Toolchain (go mod, go build)

LSP/Debug:
gopls, Delve

Qualität:
go fmt, go vet, golangci-lint

Testing:
go test, Benchmarks

C/C++ und Rust

Native Systementwicklung

- C/C++: GCC/Clang, CMake, CTest, GDB/LLDB
- Rust: rustup, Cargo, rustfmt, Clippy

Qualität & Sicherheit

- Sanitizer (ASan, UBSan) für C/C++ Speicherfehler
- Borrow Checker für Rust-Speichersicherheit
- Starke Typisierung und reproduzierbare Builds

WORKFLOW FOKUS

C/C++

Flexibilität durch CMake-Presets & plattformspezifische Compiler-Flags.

Rust

Einheitliches Ökosystem durch Cargo. Fokus auf "Safety by Design".

Shell-Skripte und SQL

Bash Automatisierung

- ShellCheck (Linting)
- shfmt (Formatting)
- Bats-core (Testing)

```
shellcheck script.sh
```

PowerShell Stacks

- PSScriptAnalyzer
- Pester (Testing)
- pwsh 7+ Runtime

```
Invoke-Pester -CI
```

SQL Engineering

- Flyway (Migration)
- SQLFluff (Linting)
- dbt (Transformation)

```
sqlfluff lint .
```

Docker: reproduzierbare Container

Build-Ökosystem

Docker CLI, Compose, Buildx und BuildKit für effiziente, parallele und cachefähige Builds.

Qualitätssicherung

Hadolint für Dockerfile-Checks, Docker Scout für Vulnerability-Scans und Dive für Image-Optimierung.

- **Multi-Stage Build**
  Build-Umgebung von Runtime trennen
- **Security Scanning**
  CVEs frühzeitig in CI erkennen
- **Registry Delivery**
  Signierte Tags & Provenance-Attests

Kubernetes: kontrolliertes Deployment

Manifest-Management

- Helm: Paketierung & Templating
- Kustomize: Overlay-basierte Anpassung
- Kubeconform: Schema-Validierung
- KubeLinter: Best-Practice-Prüfung

Cluster & Diagnose

- kind: Lokale K8s-Cluster in Docker
- kubectl: Zentrale Kommandozeile
- Stern: Parallele Log-Ansicht
- Skaffold: Entwicklungs-Workflow

Best Practices

- Ressourcen: Requests & Limits setzen
- Probes: Liveness & Readiness
- Security: Non-Root & Read-Only FS
- Rollouts: Deklarativ über GitOps

Beispiel: Go in GitHub Actions

```yaml
name: Go CI
jobs:
  test:
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version-file: go.mod
          cache: true
      - run: gofmt -l .
      - run: go vet ./...
      - run: go test ./... -race
      - run: go build ./...
```

Versions-Pinnen

Verwendung von `go.mod` stellt sicher, dass CI dieselbe Version wie die Entwicklung nutzt.

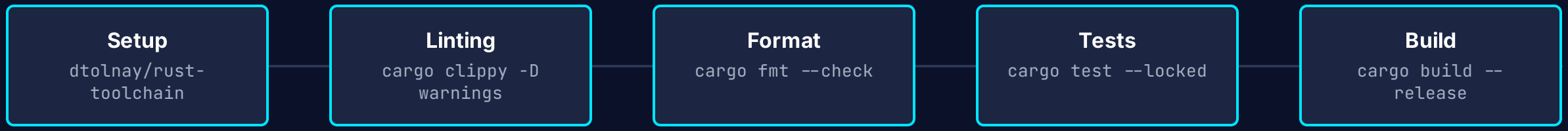
Race Detection

Das `-race` Flag erkennt Nebenläufigkeitsprobleme frühzeitig in der Pipeline.

Caching

Integriertes Modul-Caching beschleunigt wiederholte CI-Läufe erheblich.

Beispiel: Rust in GitHub Actions



Reproduzierbarkeit durch **Cargo.lock** und Pinnen der Toolchain.
Caching mit **rust-cache** beschleunigt Kompilierung massiv.

Best Practices und nächste Schritte

Do's

Pinning: Versionen & Toolchains im Repository festschreiben.

Automatisierung: Linting & Formatierung als Gate in jedem PR.

Security: Secrets & Produktionsumgebungen strikt schützen.

Don'ts

Stock Boilerplates: Kerntools von inoffiziellen Vorlagen unterscheiden.

Manual Steps: Manuelle Deployments ohne Review & CI vermeiden.

Secret Leaks: Zugangsdaten nie in Docker-Layer oder Git einbetten.